Handwritten Digit Recognition with a Neural Network Built from Scratch in NumPy

Maximiliano Andrés

github.com/Maxi-Andres/Neural-Network

Abstract. I implement a fully connected neural network for handwritten digit classification using only NumPy: forward propagation, cross-entropy loss, backpropagation and gradient descent are all derived and written by hand, without any deep learning framework. The network (784–256–128–10, ReLU and softmax, 235,146 parameters) reaches **96.96% accuracy** on a held-out MNIST test set of 23,334 images, close to scikit-learn's `MLPClassifier` with the same architecture (97.58%). I verify the hand-written gradients with a numerical gradient check, which confirms five of the six gradients to a relative error below 10^{-9} and exposes an error in the bias gradient of the first layer. I also show that the strong overfitting observed during the experiments was caused by an implementation detail, a random seed reset inside the training loop that makes every iteration use the same batch, and that batch size is therefore the main factor behind the differences between the 18 trained models. Finally, I place this kind of model within machine learning and compare it with large language models such as the one behind Claude Code.

1 Introduction

Recognizing handwritten digits is the classic first problem of machine learning. It is easy to state, a person solves it instantly, and yet writing explicit rules for it ("a 7 has a horizontal stroke on top and a diagonal one below") quickly becomes impossible because handwriting varies so much. Neural networks solve it by *learning* the rules from examples.

Modern libraries make training a network a matter of a few lines, and hide all of the mathematics. The goal of this project is the opposite: to build the network from the ground up, so that every step (how a prediction is computed, how the error is measured and how each of the 235,146 parameters is corrected) is explicit code that can be read, checked and modified. To judge the result I compare it with scikit-learn's implementation of the same architecture [1].

The contributions of this work are:

- A NumPy implementation of a 3-layer fully connected network with the complete derivation of its gradients (Section 4).
- An evaluation of 18 trained models on the full training and test sets, a per-digit error analysis and a test on digits drawn by hand (Section 5).
- A numerical gradient check of the implementation and an analysis of how the training procedure affected the results (Section 6).
- A short explanation of where this model sits within artificial intelligence and how it relates to large language models (Section 2).

2 Background: what kind of AI is this?

Artificial intelligence is the broad goal of making computers perform tasks that seem to require intelligence. *Machine learning* is the part of AI where the computer is not given rules but learns them from data, and *deep learning* is the part of machine learning that uses neural networks with several layers [2]. This project sits inside all three, and more precisely it is:

Table 1: The digit classifier of this project compared with a large language model.

	This project	**Large language model (e.g. Claude)**
Task	Classify a 28×28 image into one of 10 digits	Generate text, predicting one token (a piece of a word) at a time
Input	Fixed size: 784 pixel values	Variable length: up to hundreds of thousands of tokens
Output	Softmax over 10 classes	Softmax over a vocabulary of tens of thousands of tokens or more
Architecture	Multilayer perceptron, 3 layers	Transformer [3]: many layers that combine *attention* with MLP blocks
Training data	46,666 labeled images	A very large text corpus, then further training with human and AI feedback
Learning	Supervised	Self-supervised pretraining (predict the next token), then fine-tuning
Parameters	235,146	Billions (Claude's exact size is not public)
Training cost	Under an hour on a regular CPU	Large clusters of specialized hardware

- **Supervised learning:** the network learns from examples that come with the correct answer (an image of a 7 together with the label "7"). The other main branches are unsupervised learning (finding structure without labels) and reinforcement learning (learning from rewards).
- **Classification:** the output is one category out of a fixed set, the ten digits. The model is *discriminative*: it learns to tell classes apart, not to create new images.
- **A feedforward, fully connected network**, also called a *multilayer perceptron* (MLP): information flows in one direction, from the pixels to the answer, and every neuron of a layer is connected to every neuron of the next one.

With two hidden layers, this network is shallow by today's standards, but it already contains all the basic pieces of deep learning: parameters, nonlinear activations, a loss function, backpropagation and gradient descent.

2.1 Comparison with large language models

Claude Code is a coding assistant built on Claude, a *large language model* (LLM). It is useful to compare the two, because they look completely different but share the same foundations (Table 1).

The similarities are deeper than the table suggests. Each layer of a Transformer contains a feed-forward block that is an MLP like the one in this project: a matrix product, a nonlinearity and another matrix product. An LLM also ends with a softmax that turns scores into probabilities, only over words instead of digits, and it is also trained by minimizing the cross-entropy loss with gradients computed by backpropagation. The difference is mostly in scale and in the attention mechanism, which lets the model relate every word of a text to every other word. Claude Code then adds a layer on top that has nothing to do with training: it runs the model in a loop and lets it use tools, such as reading files and running commands, to complete programming tasks.

3 Data

I use **MNIST** [4], a dataset of 70,000 grayscale images of handwritten digits, each 28×28 pixels, with pixel intensities from 0 (black) to 255 (white) and a label from 0 to 9. The data is downloaded from OpenML through scikit-learn. Each image is flattened into a vector of 784 values and divided by 255, so all inputs lie in $[0, 1]$. This keeps the activations small and prevents numerical overflow in the exponentials of the softmax.

The dataset is split at random into a **training set** of 46,666 images (2/3) and a **test set** of 23,334 images (1/3), with a fixed seed so that the split is the same in every experiment. The network never sees the test set during training, so accuracy on it measures how well the model works on new data. The repository also contains the Kaggle version of MNIST and EMNIST [5], which were used in the scikit-learn experiments.

4 Method

4.1 Architecture

The network has an input layer of 784 values, two hidden layers of 256 and 128 neurons, and an output layer of 10 neurons, one per digit:

```
784 (pixels) → 256 (ReLU) → 128 (ReLU) → 10 (softmax)
```

The hidden layers use the rectified linear unit $\mathrm{ReLU}(z) = \max(0, z)$ [6]. It is cheap to compute and its derivative is 1 for every active neuron, so gradients do not shrink as they travel back through the layers. The output layer uses the softmax function, which turns the 10 output scores into probabilities that are positive and add up to 1:

$$\mathrm{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{10} e^{z_j}} \tag{1}$$

The predicted digit is the one with the highest probability. Counting weights and biases, the network has $784 \cdot 256 + 256 + 256 \cdot 128 + 128 + 128 \cdot 10 + 10 = 235\,146$ parameters.

4.2 Forward propagation

The images of a batch are stored as the columns of a matrix $X \in \mathbb{R}^{784 \times m}$, where m is the batch size, so the whole batch is processed with three matrix products:

$$\begin{aligned} Z_1 &= W_1 X + b_1, \quad & A_1 &= \mathrm{ReLU}(Z_1) \\ Z_2 &= W_2 A_1 + b_2, \quad & A_2 &= \mathrm{ReLU}(Z_2) \\ Z_3 &= W_3 A_2 + b_3, \quad & A_3 &= \mathrm{softmax}(Z_3) \end{aligned} \tag{2}$$

where $W_1 \in \mathbb{R}^{256 \times 784}$, $W_2 \in \mathbb{R}^{128 \times 256}$ and $W_3 \in \mathbb{R}^{10 \times 128}$ are the weights, and the biases b_1, b_2, b_3 are column vectors added to every column. Column i of A_3 holds the 10 probabilities predicted for image i.

4.3 Loss function

The labels are written in *one-hot* form: a matrix $Y \in \mathbb{R}^{10 \times m}$ whose column i has a 1 in the row of the correct digit and 0 elsewhere. The error of the network is measured with the cross-entropy loss

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{10} Y_{ki} \log \left(A_3\right)_{ki}, \tag{3}$$

which is the average of $-\log p$, where p is the probability the network gave to the correct digit. It is 0 when the network is certain and right, and it grows without limit when the network is confident and wrong.

4.4 Backpropagation

Improving the network requires the derivative of $\mathcal{L}$ with respect to every parameter. Backpropagation [7] computes all of them with the chain rule, starting from the output and moving backwards one layer at a time. For softmax followed by cross-entropy, the derivative with respect to the output scores simplifies to a remarkably simple expression:

$$\frac{\partial \mathcal{L}}{\partial Z_3} = \frac{1}{m}(A_3 - Y), \tag{4}$$

that is, the predicted probabilities minus the correct answer. Writing $dZ_3 = A_3 - Y$ (the factor $\frac{1}{m}$ is applied to the weight and bias gradients), the chain rule gives, layer by layer:

$$\begin{aligned}
dZ_3 &= A_3 - Y, & dW_3 &= \frac{1}{m} dZ_3 A_2^T, & db_3 &= \frac{1}{m} \sum_{i=1}^{m} (dZ_3)_{:,i} \\
dZ_2 &= (W_3^T dZ_3) \odot \mathrm{ReLU}'(Z_2), & dW_2 &= \frac{1}{m} dZ_2 A_1^T, & db_2 &= \frac{1}{m} \sum_{i=1}^{m} (dZ_2)_{:,i} \\
dZ_1 &= (W_2^T dZ_2) \odot \mathrm{ReLU}'(Z_1), & dW_1 &= \frac{1}{m} dZ_1 X^T, & db_1 &= \frac{1}{m} \sum_{i=1}^{m} (dZ_1)_{:,i}
\end{aligned} \tag{5}$$

where $\odot$ is element-wise multiplication and $\mathrm{ReLU}'(z)$ is 1 if $z > 0$ and 0 otherwise. The term $W_{l+1}^T dZ_{l+1}$ carries the error back to the previous layer, and multiplying by ReLU' blocks it at the neurons that were inactive.

4.5 Training

The weights are initialized with random values from a normal distribution scaled by 0.01, small enough that no neuron starts saturated, and the biases with uniform random values in $[0, 1)$. Each training iteration takes a batch of m images, runs forward propagation, computes the gradients of Equation 5 and moves every parameter a small step against its gradient (gradient descent):

$$W_l \leftarrow W_l - L \cdot dW_l, \quad b_l \leftarrow b_l - L \cdot db_l, \quad l = 1, 2, 3, \tag{6}$$

where L is the learning rate. The trained parameters are saved to `.npz` files so that training can continue later from a previous model, which I did for several of them.

4.6 Interactive demo

The project includes a small drawing application built with OpenCV. The user draws a digit with the mouse on a 200×200 canvas, and the image is resized to 20×20 and centered in a 28×28 frame, the same way the MNIST digits were prepared [4], before being fed to the network.

5 Results

I trained 18 models with different learning rates (0.01 to 0.1), batch sizes (128 to 32,768) and numbers of iterations (3,000 to 10,000). Some were trained further starting from a previous model. Every model was then evaluated on the **complete** training and test sets (Figure 1, Table 2).

The best model, `params_17`, classifies **96.96%** of the test images correctly. Its accuracy on the training set is only 2 points higher, so it generalizes well to images it has never seen. Figure 2 shows how the accuracy of `params_16` (the starting point of `params_17`) grew during training: it climbs quickly during the first 500 iterations and then improves slowly, with the train and test curves staying close together.

5.1 Errors by digit

Figure 3 shows the confusion matrix of `params_17` on the test set. The easiest digits are 1 (98.90% correct), 6 (98.31%) and 0 (97.78%), and the hardest are 8 (95.59%) and 9 (95.61%). The most frequent mistakes are between digits that look alike: 4 read as 9 (39 times) and 9 read as 4 (31), 8 read as 3 (31), and 7 read as 2 (24) or as 9 (23). Figure 4 shows some of the misclassified images. Many of them are hard even for a person.

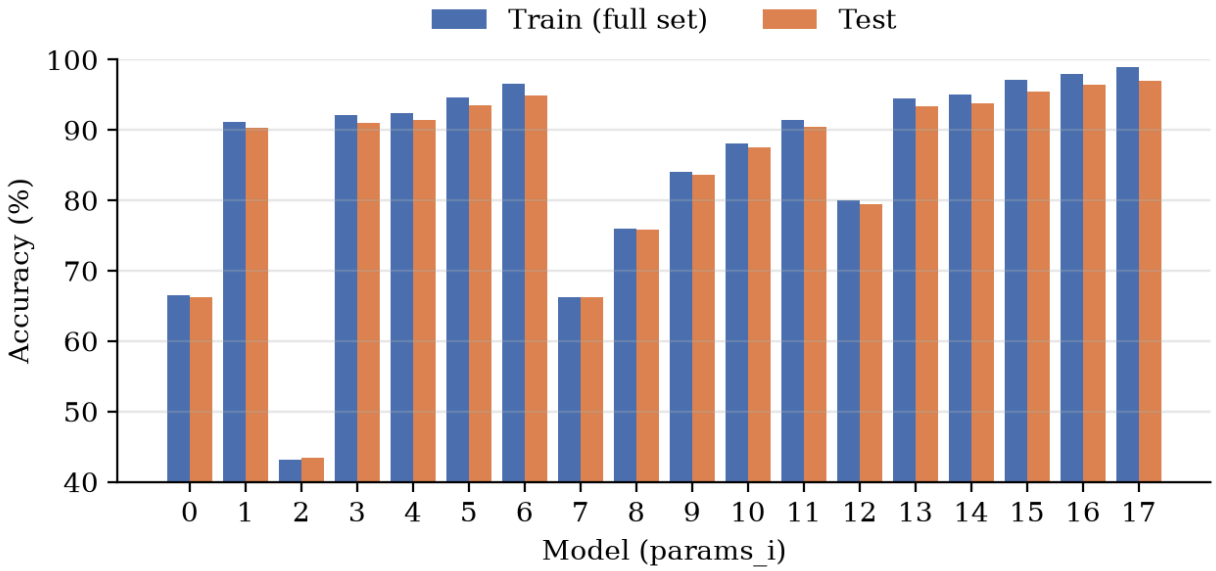


Figure 1: Accuracy of the 18 saved models on the full training set (46,666 images) and on the test set (23,334 images).

5.2 Digits drawn by hand

I also tested the models on ten digits that I drew by hand, which are not part of MNIST and are drawn with a thicker, more uniform stroke (Figure 5). `params_15` recognized all ten. `params_17`, despite its higher test accuracy, read the 1 as a 4. Ten images are too few to draw strong conclusions, but they are a reminder that accuracy on a test set only measures performance on data that looks like the training data.

5.3 Comparison with scikit-learn

I trained scikit-learn's `MLPClassifier` [1] with the same architecture (two hidden layers of 256 and 128 ReLU neurons), stochastic gradient descent, learning rate 0.1 and batches of 4,096 images on the same split. It reached **97.58%** on the test set, 0.6 points above my best model, and finished in **48 seconds** (120 passes over the training set) on the same CPU, while my best models needed around half an hour of training each.

Table 2: Models with known hyperparameters, ordered by batch size. *Batch acc.* is the accuracy on the fixed batch the model was trained on (see Section 6.2). `params_17` continues the training of `params_16`.

Model	L	Batch	Iterations	Batch acc.	Train acc.	Test acc.
`params_7`	0.01	128	10,000	100.00%	66.31%	66.28%
`params_8`	0.02	256	8,000	100.00%	76.02%	75.80%
`params_12`	0.02	512	6,000	100.00%	80.08%	79.51%
`params_9`	0.02	1,024	4,000	99.12%	84.03%	83.62%
`params_10`	0.03	2,048	3,000	96.73%	88.09%	87.49%
`params_11`	0.03	4,096	6,000	98.90%	91.48%	90.49%
`params_13`	0.03	8,192	8,000	98.77%	94.55%	93.39%
`params_14`	0.08	8,192	4,000	99.73%	95.00%	93.84%
`params_15`	0.1	16,384	4,000	99.49%	97.07%	95.52%
`params_16`	0.1	32,768	4,000	98.60%	97.99%	96.45%
`params_17`	0.1	32,768	+3,000	99.76%	**98.93%**	**96.96%**

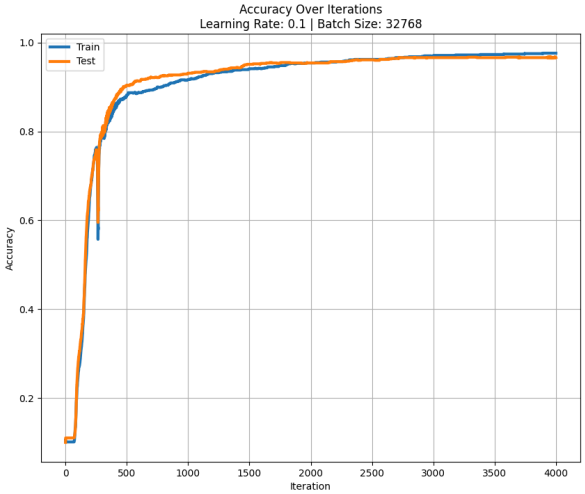


Figure 2: Accuracy during the training of `params_16` ($L = 0.1$, batch size 32,768), estimated at each iteration on random samples of 1,000 training and 1,000 test images.

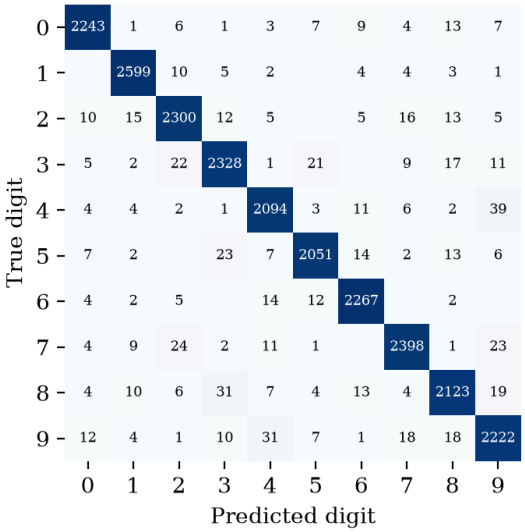


Figure 3: Confusion matrix of `params_17` on the 23,334 test images. Row: true digit. Column: predicted digit.

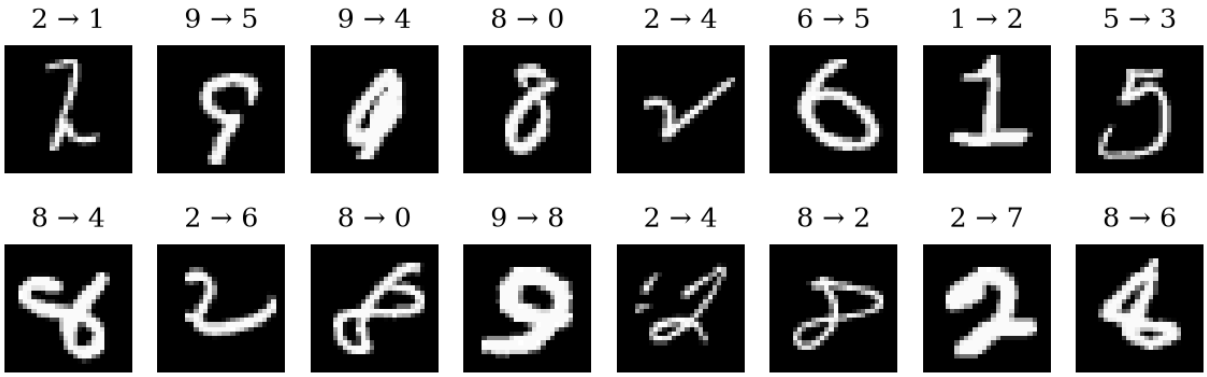


Figure 4: Test images misclassified by `params_17`, labeled *true* → *predicted*.

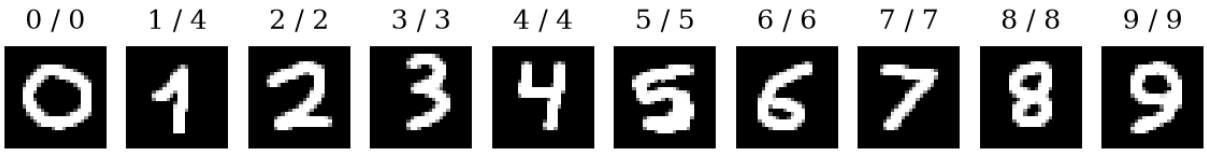


Figure 5: Digits drawn by hand, with the predictions of `params_15` / `params_17` above each image.

The difference is not that scikit-learn computes faster. Both implementations spend their time in the same NumPy matrix products, and per image processed they cost about the same: 6.7 µs for mine and 8.6 µs for scikit-learn. The difference is in *how much work* each one needs:

- scikit-learn uses gradient descent with *momentum* [8], which keeps part of the previous update and moves faster in consistent directions. It also shuffles the data every epoch, so each update sees new images, and stops automatically when the loss stops improving. In total it processed about 5.6 million images.
- My training of `params_16` processed $4{,}000 \times 32{,}768 \approx 131$ million images, 23 times more, and each iteration also spent about 0.2 s extracting the batch with pandas and estimating the accuracy, almost as much as the 0.22 s of the actual computation.

scikit-learn also adds a small L2 penalty on the weights by default, which helps it generalize.

6 Analysis

6.1 Gradient check

A hand-written backpropagation can contain errors that do not stop the network from learning, so they are hard to notice. To verify it, I compared every gradient of Equation 5 as computed by the project's code with a numerical approximation of the derivative,

$$\frac{\partial \mathcal{L}}{\partial \theta} \approx \frac{\mathcal{L}(\theta + \varepsilon) - \mathcal{L}(\theta - \varepsilon)}{2\varepsilon}, \quad \varepsilon = 10^{-5}, \tag{7}$$

for 12 parameters of each group, on 64 training images, starting from `params_16`. The relative error between both gradients should be close to zero if the code is correct (Table 3).

Five of the six gradients are correct to ten decimal places, which confirms the derivation. The bias gradient of the first layer is wrong: the code sums $W_2^T dZ_2$ instead of dZ_1, so it omits the factor $\mathrm{ReLU}'(Z_1)$ and also counts the error of inactive neurons. Using dZ_1, as in Equation 5, brings its error down to $4.6 \cdot 10^{-10}$. The error only affects 256 of the 235,146 parameters, which explains why the network still reached a high accuracy.

6.2 The fixed batch

During the experiments, the models trained with small batches looked heavily overfitted. For example, `params_7` was logged with 100% accuracy on the training data and 70% on the test data. However, its accuracy on the full training set is 66.31%, the same as on the test set.

The cause is that the training loop resets the random seed at the start of every iteration, before choosing the batch. As a result, every iteration selects *exactly the same* m images, and the model never sees the rest of the training set. With $m = 128$ the network memorizes those 128 images perfectly (the

Table 3: Relative error between the analytical and the numerical gradients.

	W_1	b_1	W_2	b_2	W_3	b_3
Project code	$8.7 \cdot 10^{-11}$	**0.52**	$2.6 \cdot 10^{-11}$	$1.7 \cdot 10^{-10}$	$1.3 \cdot 10^{-11}$	$2.2 \cdot 10^{-11}$
With fix		$4.6 \cdot 10^{-10}$				

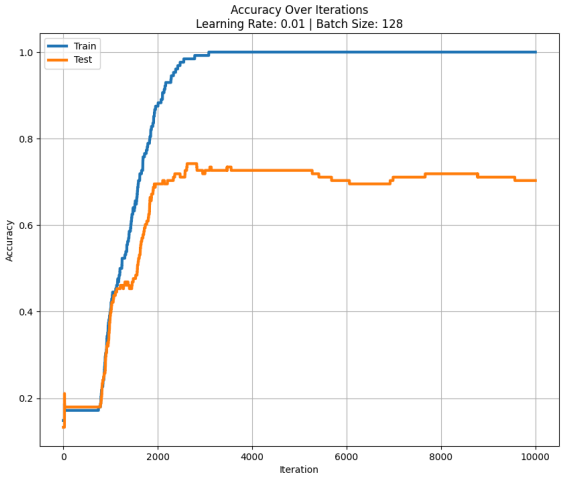


Figure 6: Training of `params_7` with a batch of 128 images. The "train" curve reaches 100% because it measures the same 128 images used for training.

Batch acc. column of Table 2) and the 100% that was logged during training was measured on them, while it performs much worse on everything else (Figure 6). In other words, the effective size of the training set was the batch size.

This explains the most visible pattern of Table 2: test accuracy grows steadily with the batch size, from 66% with 128 images to 96.5% with 32,768, regardless of the learning rate. It also means that the best models were trained on only 70% of the training set, and that the network could likely improve further.

7 Conclusions

I built a neural network from scratch, with all of its mathematics written as explicit NumPy code, and trained it to recognize handwritten digits with 96.96% accuracy on images it had never seen. A library implementation of the same architecture reached 97.58% in a fraction of the time, but as my measurements show, the gap comes from the training procedure and not from the arithmetic. Implementing everything by hand gives full control and a real understanding of what happens inside the network. It also means that errors are easy to make and hard to see: the network learned well despite a wrong gradient and a training loop that used a single batch, and only systematic checks revealed both.

The natural next steps follow from the analysis: correct the gradient of b_1, draw a new batch in each iteration (or shuffle the data every epoch), and add momentum. A network that only gets the prediction right 97% of the time also illustrates a broader point. Even far larger models, like the language models discussed in Section 2, are statistical systems that learn patterns from data, and they can be confidently wrong.

8 Reproducibility

All code, data and trained models are in the repository. `install.bat` (Windows) or `install.sh` (Linux/macOS) creates a virtual environment with every dependency, and `run.bat` / `run.sh` opens the main notebook, `Neural_Network.ipynb`, which contains the training, the evaluation and the drawing demo. The trained parameters of every model are in `Weights_&_Bias/`. The accuracies in Section 5 and Section 6 were computed on the full training and test sets, with the split described in Section 3. The timings were measured on a single consumer CPU and will vary across machines.

References

[1] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[3] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.

[4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[5] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik, "EMNIST: Extending MNIST to Handwritten Letters," in *International Joint Conference on Neural Networks (IJCNN)*, 2017, pp. 2921–2926.

[6] V. Nair and G. E. Hinton, "Rectified Linear Units Improve Restricted Boltzmann Machines," in *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 2010, pp. 807–814.

[7] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning Representations by Back-Propagating Errors," *Nature*, vol. 323, pp. 533–536, 1986.

[8] I. Sutskever, J. Martens, G. Dahl, and G. Hinton, "On the Importance of Initialization and Momentum in Deep Learning," in *Proceedings of the 30th International Conference on Machine Learning (ICML)*, 2013, pp. 1139–1147.